Recuperarea cheii private ECDSA

Tool generic: Nonce Reuse + Lattice Attack

Aplicatie: MITRE ICCS 2012 Multi-Day Challenge

Writeup – varianta extinsa

Contents

1 Contextul problemei si strategia generala **2**

2 Fundamente matematice ECDSA **2**

3 Atacul 1: Nonce Reuse (recuperare exacta) **2**
3.1 Conditia de vulnerabilitate . . . 2
3.2 Derivarea formulei de recuperare . . . 3
3.3 Implementare . . . 3

4 Atacul 2: Lattice Attack pe HNP (recuperare aproximativa) **4**
4.1 Hidden Number Problem (HNP) . . . 4
4.2 Reducere la o retea (lattice) . . . 4
4.3 Implementare . . . 4

5 Parsare generica a semnaturilor **5**

6 Aplicare pe challengeul MITRE ICCS 2012 **6**
6.1 Parametrii curbei . . . 6
6.2 Analiza semnaturilor . . . 6
6.3 Verificare finala . . . 6

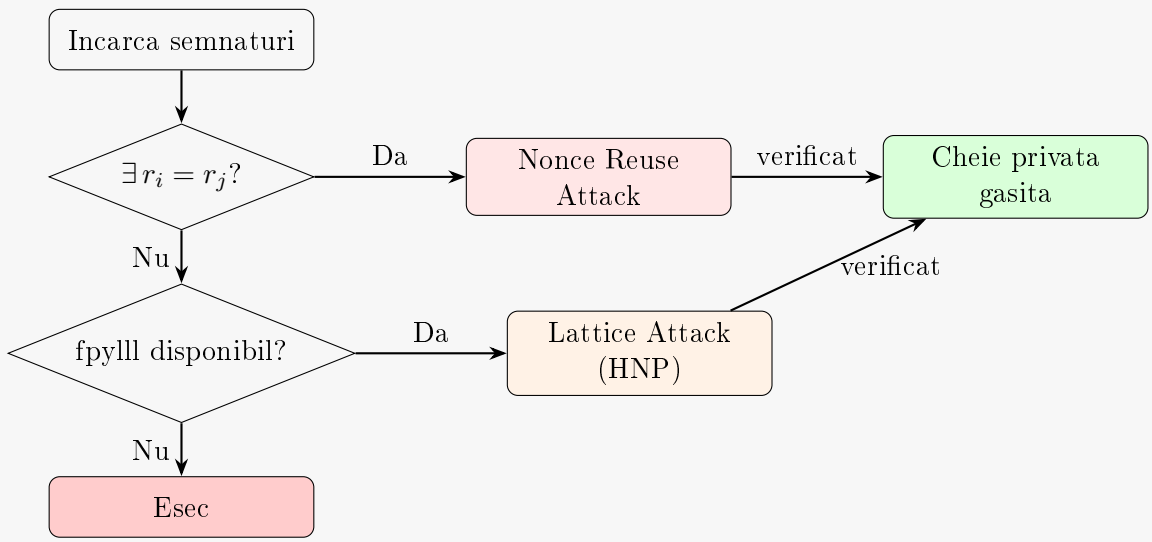
7 Extinderi si limitari **6**
7.1 Cand functioneaza fiecare atac . . . 6
7.2 Parametri configurabili . . . 6
7.3 Lectii de securitate . . . 7

Contextul problemei si strategia generala

Scriptul `solve_ecdsa_strong.py` este un **tool generic de recuperare a cheii private ECDSA**, proiectat sa functioneze pe orice set de semnaturi, nu doar pe challengeul MITRE. El implementeaza doua atacuri distincte, aplicate in cascada:

1. **Nonce Reuse Attack** — detectie si exploatare automata a reutilizarii nonce-ului efemer k (cazul cel mai comun si complet exploatabil);
2. **Lattice Attack (HNP)** — recuperare bazata pe reducerea retelelor, aplicabila cand nonce-ul are biti cunoscuti/slabi, chiar daca nu e reutilizat.

Fluxul de decizie al scriptului



Fundamente matematice ECDSA

Definitie 1 (Schema de semnatura ECDSA)**.** Fie p prim, $E(\mathbb{F}_p) : y^2 = x^3 + ax + b$, $G \in E(\mathbb{F}_p)$ de ordin prim q, cheia privata $d \in \{1, \ldots, q-1\}$, cheia publica $Q = dG$.

Semnare: Pentru mesaj cu hash m, se alege $k \xleftarrow{\$} \{1, \ldots, q-1\}$:

$$r = x_{kG} \bmod q, \qquad s = k^{-1}(m + rd) \bmod q.$$

Semnatura: (r, s). Daca $r = 0$ sau $s = 0$, se reia cu alt k.

Verificare: Se calculeaza $u_1 = ms^{-1} \bmod q$, $u_2 = rs^{-1} \bmod q$ si se verifica $x_{u_1G+u_2Q} \equiv r \pmod q$.

Observatie 1. Securitatea ECDSA depinde *critic* de unicitatea si aleatoarea lui k. Orice slabiciune in alegerea k este exploatabila.

Atacul 1: Nonce Reuse (recuperare exacta)

Conditia de vulnerabilitate

Daca acelasi k este folosit pentru doua mesaje m_1, m_2, atunci x_{kG} este identic, deci:

$$r_1 = r_2 = r.$$

Aceasta este conditia *necesara si suficienta* detectata automat de script prin gruparea semnaturilor dupa valoarea r.

Derivarea formulei de recuperare

Avem sistemul:

$$s_1 = k^{-1}(m_1 + rd) \bmod q \tag{1}$$
$$s_2 = k^{-1}(m_2 + rd) \bmod q \tag{2}$$

Impartind (1) la (2):

$$\frac{s_1}{s_2} = \frac{m_1 + rd}{m_2 + rd} \pmod q \quad \Longrightarrow \quad s_1(m_2 + rd) = s_2(m_1 + rd) \pmod q$$

$$d \cdot r(s_1 - s_2) = s_2 m_1 - s_1 m_2 \pmod q$$

Formula cheia privata (nonce reuse)

$$\boxed{d \;=\; \frac{m_2 s_1 - m_1 s_2}{r(s_2 - s_1)} \bmod q}$$

Este de observat ca se pot recupera si $k_1 = k_2 = k$:

$$k = \frac{m_1 - m_2}{s_1 - s_2} \bmod q$$

lucru implementat in `recover_nonce_k()`, util pentru audit sau validare.

Implementare

```python
def group_by_r(sigs):
    g = defaultdict(list)
    for s in sigs:
        g[s["r"]].append(s)
    return g

def detect_reuse(groups):
    return [g for g in groups.values() if len(g) > 1]

def recover_private_key(m1, m2, r, s1, s2, q):
    num = (m2 * s1 - m1 * s2) % q
    den = (r  * (s2 - s1))    % q
    return num * pow(den, -1, q) % q
```

Listing 1: Detectie automata si exploatare nonce reuse

Atacul 2: Lattice Attack pe HNP (recuperare aproximativa)

Hidden Number Problem (HNP)

Chiar daca nonce-ul k nu este reutilizat, el poate fi *slab* — de exemplu, daca primii ℓ biti sunt cunoscuti sau sunt zero (MSB bias). Aceasta situatie se modeleaza ca **Hidden Number Problem (HNP)**:

Definitie 2 (HNP in context ECDSA)**.** Dat find n perechi (t_i, u_i) cu:

$$t_i = r_i s_i^{-1} \bmod q, \qquad u_i = m_i s_i^{-1} \bmod q,$$

se stie ca $k_i = t_i d + u_i \bmod q$ are primii ℓ biti egali cu zero. Se cere recuperarea lui d.

Reducere la o retea (lattice)

Construim o matrice $B \in \mathbb{Z}^{(n+1)\times(n+1)}$:

$$B = \begin{pmatrix} q & 0 & \cdots & 0 & 0 \\ 0 & q & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & q & 0 \\ t_1 & t_2 & \cdots & t_n & K \end{pmatrix}$$

unde $K = 2^\ell$. Vectorul solutie $\mathbf{v} = (k_1 \bmod q, \ldots, k_n \bmod q, Kd/q)$ este un vector scurt in reteaua generata de coloanele lui B.

Aplicand **algoritmul LLL** (reducerea retelei), gasim vectori scurti, printre care, cu probabilitate buna, se afla vectorul ce contine d.

Observatie 2. Algoritmul LLL garanteaza gasirea unui vector de lungime cel mult $2^{(n-1)/4} \cdot \det(B)^{1/n}$ fata de cel mai scurt vector teoretic. In practica, pentru $n \geq 4$ semnaturi si $\ell \geq 8$ biti slabi, rata de succes este ridicata.

Implementare

```python
def lattice_attack(signatures, q, known_bits=8):
    n = len(signatures)
    B = IntegerMatrix(n+1, n+1)
    K = 2 ** known_bits

    for i, sig in enumerate(signatures):
        m, r, s = sig["m"], sig["r"], sig["s"]
        s_inv = pow(s, -1, q)
        t = (r * s_inv) % q
        B[i, i] = q
        B[i, n] = t

    for j, sig in enumerate(signatures):
        m, r, s = sig["m"], sig["r"], sig["s"]
        s_inv = pow(s, -1, q)
        u = (m * s_inv) % q
```

```
        B[n, j] = u

    B[n, n] = K
    LLL.reduction(B)        # reducere LLL in-place

    for row in B:
        d = row[n] % q
        if d != 0:
            return d        # candidat pentru cheia privata
    return None
```

Listing 2: Constructia matricei si reducerea LLL (via fpylll)

Dependenta: modulul `fpylll` (Python wrapper pentru FPLLL). Scriptul verifica disponibilitatea la import si trece automat la fallback daca nu este instalat.

Parsare generica a semnaturilor

Scriptul suporta doua formate de intrare, detectate automat dupa extensia fisierului:

```
def parse_signatures_auto(path):
    if path.endswith(".json"):
        with open(path) as f:
            data = json.load(f)
        return [
            {"m": int(x["hash"], 16),
             "r": int(x["r"],    16),
             "s": int(x["s"],    16)}
            for x in data
        ]

    # Format text (ex: signatures.txt)
    with open(path, "r", encoding="utf-8") as f:
        txt = f.read()
    matches = re.findall(
        r"Hash:\s*([0-9A-Fa-f]+).*?R:\s*([0-9A-Fa-f]+).*?S:\s*([0-9
            A-Fa-f]+)",
        txt, re.S
    )
    return [{"m": int(h,16), "r": int(r,16), "s": int(s,16)}
            for h, r, s in matches]
```

Listing 3: Parser multi-format pentru semnaturi

Format JSON asteptat:

```
[
  {"hash": "DE37B3...903", "r": "ACB2C1...0E2", "s": "BE4FA9...4B8
      "},
  {"hash": "28469B...BAA", "r": "ACB2C1...0E2", "s": "D3540E...7E2
      "}
]
```

Aplicare pe challengeul MITRE ICCS 2012

Parametrii curbei

Parametru	Valoare
p	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFE56D
Curba	$E : y^2 \equiv x^3 + 5 \pmod p$
q	01000000000000000000000000001DCE8D2EC6184CAF0A971769FB1F7

Analiza semnaturilor

```
[*] Running FULL AUTO attack
[+] Reuse detected: r1 == r2
[*] Trying nonce-reuse exploit...
[checkmark] PRIVATE KEY (reuse)
8E88B0433C87D1269173487795C81553AD819A1123AE54854B3C0DA7

Lattice: skipped (nonce reuse succeeded)
```

Nonce reuse-ul este detectat imediat: cele doua semnaturi au $r_1 = r_2$, deci atacul direct produce cheia privata fara a fi nevoie de lattice.

Verificare finala

Verificarea $d \cdot G = Q$ confirma corectitudinea:

$$d \cdot G \;=\; Q \quad \checkmark$$

Cheie privata recuperata (hexazecimal)

8E88B0433C87D1269173487795C81553AD819A1123AE54854B3C0DA7

Extinderi si limitari

Cand functioneaza fiecare atac

Situatie	Atac	Conditie
k reutilizat complet	Nonce Reuse	$r_i = r_j$, 2 semnaturi
k cu MSB-uri zero	Lattice (HNP)	$\ell \geq 2$, $n \geq 4$ semn.
k cu LSB-uri cunoscute	Lattice (HNP)	varianta extinsa a matricei
k complet aleator	Nici unul	securitate standard

Parametri configurabili

- `MAX_LATTICE_SIGS` (implicit 8): numarul de semnaturi folosite in constructia retelei. Valori mai mari cresc probabilitatea de succes, dar si timpul de calcul (LLL este $O(n^3)$ per iteratie).
- `KNOWN_BITS_DEFAULT` (implicit 8): numarul de biti slabi ℓ presupusi. Trebuie adaptat la sursa nonce-ului.

Lectii de securitate

1. **RFC 6979** este raspunsul corect: nonce determinist, derivat din (m, d) via HMAC-DRBG, elimina orice dependenta de surse aleatoare.
2. O singura reutilizare a nonce-ului expune *complet* cheia privata, indiferent de dimensiunea curbei sau de puterea algoritmului de hash.
3. Atacuri reale celebre: *Sony PlayStation 3* (2010) — nonce constant; *Android Bitcoin wallets* (2012) — PRNG nesecurizat pe `/dev/urandom`.
4. Lattice attacks sunt fezabile in practica chiar si cu slabiciuni de $\ell = 4$ biti, cu un numar suficient de semnaturi.